



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第七章 二维码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

条形码简史

20世纪20年代, 约翰·科芒德实现邮政单据自动分拣方法。

1959年, 吉尔·德塞伊赛尔申请了用条形码表示数字的专利。

70年代, UPC码 (Universal Product Code) 在北美超市得到广泛应用。

1997、2000年确定了国家标准。

1920195019591960197019882000

1949年, 美国乔·伍德兰德、伯尼·西尔沃发明了第一个条形码专利。

60年代, 西尔沃尼亚发明一套条形码系统并在北美铁路系统成功应用。

1988年, 中国成立了中国物品编码中心, 80年代末国外研制出二维码。

Page 208



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

二维码概念

二维码 (2-dimensional bar code), 也叫二维条码或二维形码, 它是用某种特定的几何图形按一定规律在平面 (二维方向上) 分布的黑白相间的记录数据符号信息的图形。它在代码编制上巧妙地利用构成计算机内部逻辑基础的0、1比特流的概念, 使用若干与二进制相对应的几何形体来表示文字数值信息, 通过图像输入设备或光电扫描设备自动识读以实现信息自动处理。

它的起源是日本, 原本是DensoWave公司为了追踪汽车零部件而设计的一种条码。



Page 209



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

二维码的分类与原理

二维码可以分为行排式二维码和矩阵式二维码。

二维码

行排式二维码

Code 16K
PDF417
.....

矩阵式二维码

Code One
QR Code
.....



PDF417



Code One

Page 210



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

行排式二维码

行排式二维码形态上是由多行短截的一维条码堆叠而成。

行排式二维码, 又称堆积式二维码或层排式二维码, 其编码原理是建立在一维条码基础之上, 按需要堆积成二行或多行。

它在编码设计、校验原理、识读方式等方面继承了一维条码的一些特点, 识读设备和条码印刷与一维条码技术兼容。但由于行数的增加, 需要对行进行判定, 其译码算法与软件也不完全相同于一维码。

有代表性的行排式二维码有Code 49、Code 16K、PDF417等。



Code 49



Code 16K



PDF417

Page 211



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

矩阵式二维码

矩阵式二维码以矩阵的形式组成。

矩阵式二维码, 又称棋盘式二维码, 它是在一个矩形空间通过黑、白像素在矩阵中的不同分布进行编码。

在矩阵相应元素位置上, 用点 (方点、圆点或其他形状) 的出现表示二进制“1”, 点的不出现则表示二进制“0”, 点的排列组合确定了矩阵式二维码所代表的意义。矩阵式二维码是建立在计算机图像处理技术、组合编码原理等基础上的一种新型图形符号自动识读处理码制。

有代表性的矩阵式二维码有Code One、QR Code、Data Matrix等。



Code One



1234567890123456789012
Data Matrix



QR Code

Page 212

1

- **可靠性强**
条形码的读取准确率远远超过人工记录，平均每15000个字符才会出现一个错误。
- **效率高**
条形码的读取速度很快，相当于每秒40个字符。
- **易于制作**
条形码的编写很简单，制作也仅仅需要印刷，被称为“可印刷的计算机语言”。
- **高密度**
二维码通过利用垂直方向的堆积来提高条码的信息密度，而且采用高密度图形表示。



300个字符或数字能编进这样大小的QR码里面



同样的数据QR码只有条码空间的1/11的大小

Page 213

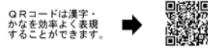
- **构造简单**
条形码识别设备的构造简单，使用方便。
- **灵活实用**
条形码符号可以手工键盘输入，也可以和有关设备组成识别系统实现自动化识别，还可以和其他控制设备联系起来实现整个系统的自动化管理。
- **成本低**
与其它自动化识别技术相比较，条形码技术仅仅需要一小张贴纸和相对构造简单的光学扫描仪，成本相当低廉。
- **纠错功能**
二维码不仅能防止错误，而且能纠正错误，即使条形码部分损坏，也能将正确的信息还原出来。



QR码损坏一部分依然能还原信息

Page 214

- **多语言形式，可表示图像**
二维码具有字节表示模式，即提供了一种表示字节流的机制。不论何种语言文字它们在计算机中存储时以机内码的形式表现，而内部码都是字节码，可识别多种语言文字的条码。
 - **具有加密体制**
可以先用一定的加密算法将信息加密，再用二维码表示在识别二维码时，再加以一定的解密算法，便可以恢复所表示的信息。
- 除此之外，二维码还具有**可表示信息量密度高、尺寸大小比例可变**、**可以使用激光或CCD阅读器方便识读**等众多优点。



QR码能够有效地表示中国汉字、日本汉字

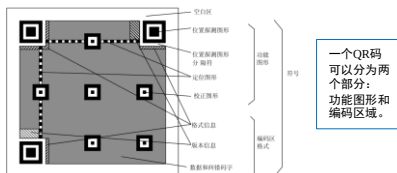
Page 215

- 目前二维码方面流行三大国际标准：
- PDF417：不支持中文；
 - DM（Data Matrix）：专利未公开，需要支付专利费用；
 - QR Code：专利公开，支持中文。

QR Code（全称Quick Response Code）是由日本Densyo公司于1994年9月研制的一种矩阵式二维条码，它具有以下特点：

1. 高密度编码，信息容量大
2. 编码范围广
3. 容错能力强，具有纠错功能
4. 译码可靠性高
5. 可引入加密措施
6. 成本低，易制作，持久耐用
7. 条码符号形状、尺寸大小比例可变
8. 二维条码可以使用激光或CCD阅读器识读，QR Code具有全方位（360°）识读特点
9. 能够有效地表示中国汉字

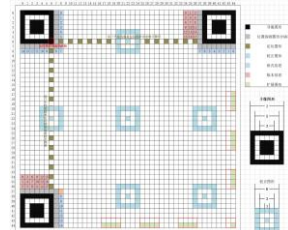
Page 216



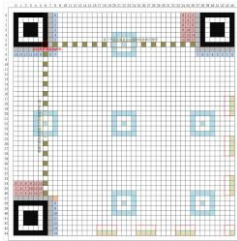
QR码共有40个版本（版本1~版本40），其中版本1的模块数为21*21，而每增加一个版本，其每边的模块数各增加4。因此版本40的模块数为177*177。

Page 217

版本7的模块数为45*45。
以版本7为例，介绍QR码各个部分的结构。



Page 218



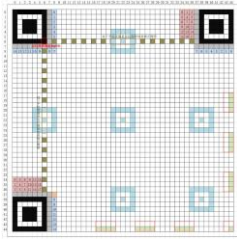
寻像图形、位置探测图形：
协助扫描软件定位QR码并转换坐标系。

寻像图形包括三个相同的位置探测图形，分别位于图形中的左上角、右上角和左下角。每个位置探测图形由7*7个模块组成。如图，寻像图形为黑色区域。

位置探测图形分隔符：
区分功能图形和编码区域。

每个位置探测图形和编码区域之间有宽度为1个模块的分隔符。位置探测图形分隔符为灰色区域，此区域应为空白。

Page 219



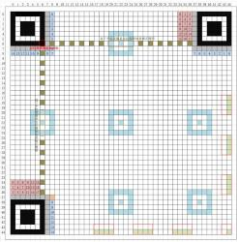
定位图形：
确定符号的密度和版本，提供决定模块坐标的基准位置。

定位图形为图中的茶色区域。水平和垂直的定位图形分别始于第6行和第6列。

此外，坐标 (6,6) 同时是位置探测图形和定位图形，且为定位图形的开始坐标。第6行和第6列中的6个水绿色部分也是定位图形。

定位图形是由深色与浅色模块交替组成，开始与结尾都是深色模块。

Page 220



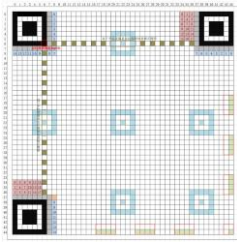
校正图形：
在图像有一定程度损坏的情况下，译码软件可以通过它同步图像模块的坐标映像。

校正图形的数量视符号和版本号而定，版本1没有校正图形，版本2及以上均含有校正图形。

如图，版本7中，校正图形为水绿色区域。

校正图形为固定的参照图形，每个校正图形由5*5的模块组成。可以通过中心模块行列坐标值推出校正图形的具体位置。

Page 221



格式信息：
存放纠错等级和掩模信息。

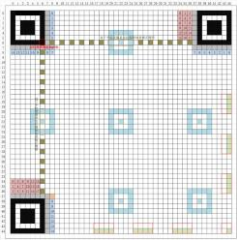
格式信息是一个15位数据，由2位纠错指示符+3位掩模图形参考+10位纠错码组成，为图中深蓝色区域。

同时，左下角格式信息编号8上方的橙色区域，此区域永远为深色模块，不用于存放任何信息。

版本信息：
用于存放QR码的版本号。

为图中红色区域，两个3*6模块。

Page 222



扩展图形：
最初的目的用于将来对QR码功能的扩展，并不用于对数据的编码。

扩展图形由位于符号右下角的一个4个模块组成的方块以及位于符号右边和下边的一些8个模块的块组成。8个模块的块的数目取决于符号的版本，计算公式为：

$8 \text{ 模块的数目} = 2 * (N \text{ DIV } 2)$
其中N为版本号，DIV表示除法运算。

扩展图形为图中红框区域，即版本7中有6个8模块组成的块和1个4模块组成的块。

Page 223

QR码的纠错能力：

级别	纠错能力
L	约7%
M	约15%
Q	约25%
H	约30%

纠错能力相当于全部码字（组成数据的单位，在QR码中，1码字相当于8比特的）恢复率。

Page 224

QR码的编码流程如下：

1. 数据分析
确定要进行编码的字符类型，选择所需的版本信息和纠错版本。
2. 数据编码
采用既定规则，将数据字符转换为位流，并加必要符号，后将位流转换为码字（每8位一个码字）。
3. 纠错编码
生成纠错码字放到数据码字后。
4. 构造最终信息
按规定将每一块中置入数据和纠错码字，必要时加剩余位。
5. 在矩阵中布置模块
将寻像图形、分隔符、校正图形与码字按规则排列，放入二维码矩阵。
6. 掩模
用8种掩模图形依次对符号的编码区域的位置图进行掩模处理，评价所得的8种结果，选择最优的一种。
7. 格式和版本信息
生成版本信息和格式信息，构成符号。

Page 225

表2：QR码可表示的数据类型及其相应的模式指示符。

表3：各版本及模式所对应的字符计数指示符的长度。

表2 模式指示符	
模式	指示符
ECI	0111
数字	0001
字母数字	0010
8位字节	0100
日本汉字	1000
中国汉字	1101
结构延伸	0011
FNC1	0101 (第一位置) 1001 (第二位置)
终止符 (信息结束)	0000

表3 字符计数指示符的长度					
版本	数字 模式	字母数字 模式	8位字节 模式	日本汉字 模式	中国汉字 模式
1~9	10	9	8	8	-
10~26	12	11	16	16	-
27~40	14	13	16	12	-

Page 226

举例说明QR码的编码过程。

- 数据分析：确定编码的字符类型、版本信息和纠错等级。

假设：
字符类型：数字模式 “01234567”
版本信息：版本1
纠错等级：H

Page 227

➤ 数据编码

- 数据字符转换为位流
首先将数字每三位分为一组，然后将每组数据转换为10位长度的二进制。如果最后一组只有两位，则使用7位长度的二进制；如果只剩一位，则使用4位长度。最后将每组的二进制连接起来形成数据位流，并在前面加上模式指示符和字符计数指示符。

- 如：01234567
1. 分组：012 / 345 / 67
 2. 转换为二进制：
012：000001100（加粗的0是为了凑够10位长度）
345：0101011001
67：1000011
 3. 数据位流：000001100 0101011001 1000011

Page 228

➤ 数据编码

- 数据字符转换为位流
- 4. 确定模式指示符和字符计数指示符
根据表2，数字模式下的模式指示符为0001；
根据表3，数字模式下版本1的字符计数指示符的位数为10。

“01234567”共有8位数字，则将位数8转换为10位长度的二进制。
8：000001000

表2 模式指示符	
模式	指示符
ECI	0111
数字	0001
字母数字	0010
8位字节	0100
日本汉字	1000
中国汉字	1101
结构延伸	0011
FNC1	0101 (第一位置) 1001 (第二位置)
终止符 (信息结束)	0000

版本	数字 模式	字母数字 模式	8位字节 模式	日本汉字 模式	中国汉字 模式
1~9	10	9	8	8	-
10~26	12	11	16	16	-
27~40	14	13	16	12	-

Page 229

➤ 数据编码

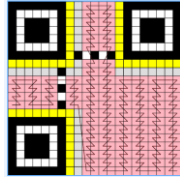
- 数据字符转换为位流
- 5. 最终位流：模式指示符+字符计数指示符+数字位流
0001 + 000001000 + 000001100 0101011001 1000011

Page 230

在矩阵中布置模块

- 右图展示了数据码字填写时的走位
- 符号字符的布置
在QR码符号的编码区域中，符号字符以2个模块宽的纵列从符号右下角开始布置，并自右向左，且交替地向下向上或从上向下安排。

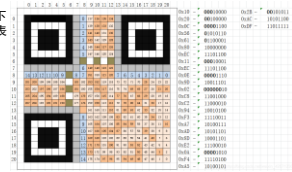
具体走位路线如右图。



Page 237

在矩阵中布置模块

在上一个步骤中，我们已经生成了最后的数据码字：
0x10 0x20 0x00 0x56 0x61 0x80 0xE0 0x11 0xE0 0x9D 0x02
0x08 0x02 0x94 0xF3 0xA7 0xA0 0x8D 0xE2 0x0A 0xF4 0xA5 0x2B
0xAC 0xDF
将这26个16进制数转换为8为的二进制数，并按照上一页的路线依次填入到对应位置。
最终结果如下图。其中浅色代表0，深色代表1。



Page 238

掩模

QR码中如果出现大面积的空白或者黑块，会导致扫描器识别困难。为了让QR图形看起来尽可能凌乱，且尽可能避免位置探测图形中的位图1011101的出现，需对QR图形进行掩模操作，步骤如下：

- 掩模不用于功能图形及格式图形，如寻像图形、定位图形、校正图形、位置探测图形分隔符、格式信息和版本信息；
- 数据码字与掩模图形进行异或操作后再进行绘制；
- 对每个结果图形的不合要求的部分计分，以评估这个结果；
- 选择得分最低的结果。

Page 239

掩模

表23给出了掩模图形的参考（放置于格式信息中的二进制参考）和掩模图形生成的条件。

掩模图形是通过编码区域（不包括格式信息和版本信息）内那些条件为真的模块定义为深色而产生的。
i代表模块的行的位置，j代表模块的列的位置，其中i和j均从0开始。

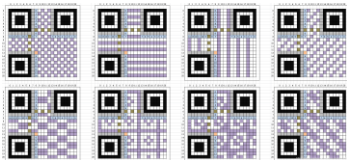
表23掩模图形的生成条件。

掩模图形参考	条件
000	$(i+j) \bmod 2 = 0$
001	$\text{row}(i) = 0$
010	$\text{row}(j) = 0$
011	$(i+j) \bmod 3 = 0$
100	$((i \bmod 2) + (j \bmod 3)) \bmod 2 = 0$
101	$(i \bmod 2) = (j \bmod 3) = 0$
110	$((i \bmod 2) + (j \bmod 3)) \bmod 2 = 0$
111	$((i \bmod 2) + (j \bmod 3)) \bmod 2 = 0$

Page 240

掩模

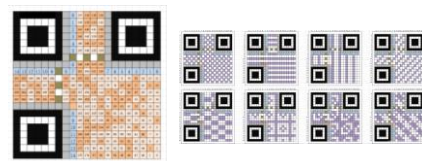
下图为版本1符号的所有掩模图形。
分别模拟了用掩模图形参考000、001、010、011、100、101、110、111的掩模结果。



Page 241

掩模

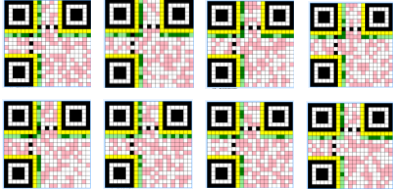
掩模数据是将左图的橙色区域的数据信息分别与右图的8种情况下的紫色部分进行异或操作得到的，掩模的最终结果在下页显示。



Page 242

掩模

下图的粉色区域分别为掩模为000、001、010、011、100、101、110、111得到的最终结果。



Page 243

格式和版本信息

计算出纠错码和格式信息

根据表25，可以得到纠错等级H的二进制指示符为10。
假设使用的掩模图形参考为011。

将2位纠错指示符和3位掩模图形参考连接在一起，得到5位的**数据码**，并使用BCH(15,5)编码计算得到**纠错码**。其中，BCH(15,5)表示BCH码的总长度为15，数据码为5位，纠错码为10位。

表25纠错等级指示符

纠错等级	二进制指示符
L	01
M	00
H	11
E	10

Page 244

格式和版本信息

计算出纠错码与格式信息

1. 连接2位**纠错指示符** + 3位**掩模图形参考**：10 + 011
2. 由于BCH码总长度为15，因此需要将10011凑够15位，在数据后方补0：100110000000000
3. 计算**纠错码**，纠错码为数据码除以生成码所得的余数，因此：假设BCH码使用的生成码为：10100110111
纠错码=100110000000000 % 10100110111，即0111000010
4. 则**最终格式信息码**：2位纠错指示符 + 3位掩模图形参考 + 10位纠错码，即
10 + 011 + 0111000010

格式信息的掩模

作用：确保纠错等级和掩模图形参考合在一起的结果不全为0。
需要将15位格式信息码与掩模图形101010000010010进行异或运算。

Page 245

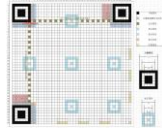
格式和版本信息

格式信息的掩模

格式信息码	100110111000010
掩模图形	101010000010010
异或结果	001100111010000

将掩模后的格式信息按照序号填入对应位置，即下图中的蓝色区域中。

对应位	14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
格式信息	0 0 1 1 0 0 1 1 1 0 1 0 0 0 0



Page 246

格式和版本信息

计算版本信息

使用BCH(18, 6)编码计算得到纠错码。原理同上，BCH(18, 6)表示BCH码总长度为18位，其中数据码为6位，纠错码12位。
版本信息的计算和格式信息类似，也是使用**长除法**。只是这一次使用的生成码为：1111100100101，使用数据码除以生成码，所得余数就是纠错码。

由于版本1无需绘制版本信息，因此暂时使用版本7来进行版本信息生成的讲解。

1. 将版本号7转换为6位长度的二进制数据码
7：000111
2. 由于这里的BCH码总长度为18，需将000111凑够18位，于是在数据后方补0得到数据码：
000111000000000000
3. 纠错码=0011100000000000 % 1111100100101
即为110010010100

Page 247

格式和版本信息

计算版本信息

4. 6位数据码 + 12位纠错码即为最终版本信息码
000111 + 110010010100(0x7c94)
与格式信息不同，版本信息码生成后不再需要单独进行掩模运算。
5. 将结果分别填入左下角和右上角的版本信息中，注意顺序。数字是倒着填入的，即末尾数字对应编号0，开头数字对应编号17，并且编号是从0开始的。

对应位	17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
版本信息	0 0 0 1 1 1 1 1 0 0 1 0 0 1 0 1 0 0

Page 248

最终结果：



识别步骤

1. 条码定位
2. 条码分割
3. 解码



条码定位

条码的定位就是找到条码符号的图像区域，对有明显条码特征的区域进行定位，然后根据不同条码的定位图形结构特征对不同的条码符号进行下一步的处理。



条码定位

定位采用以下步骤：

1. 利用点运算的阈值理论将采集到的图像变为二值图像，即对图像进行二值化处理；
2. 得到二值化图像后，对其进行膨胀运算；
3. 对膨胀后的图像进行边缘检测得到条码区域的轮廓。



条码定位

4. 确定寻像图形
5. 探测图形中心坐标
6. 确定两个举例
7. 确定版本号
8. 构造位图
9. 得到纠错等级和掩模图形



条码分割

分割采用以下步骤：

1. 将原图像按比例缩小进行分割，计算其特征值；
2. 分块继承父块纹理类别，结合其周围纹理类型进行修正；
3. 重复步骤二直至图像被划分为2*2大小，分割结束；
4. 分割结束后，图中可能出现的孤立的小区域可作为噪声删除。

> 解码

得到一幅标准的条码图像后，对该符号进行网络取样，对网格每一个文点上的图像像素取样，并根据阈值确定是深色块还是浅色块。构造一个位图，用二进制的“1”表示深色像素，“0”表示浅色像素，从而得到条码的原始二进制序列列值，然后对这些数据进行纠错和译码：

1. 异或处理 (XOR)
2. 确定符号码字
3. 重新排列码字序列
4. 执行错误检测和纠错译码程序

最后根据条码的逻辑编码规则把这些原始的数据位流转换成数据码字。

Page 255

> 食品追溯

原材料供应商在向食品厂家提供原材料时候进行批次管理，将原材料的原始生产数据制造日期、食用期限、原产地、生产者、遗传基因组合等信息录入到二维码中，并打印带有二维码的标签，粘贴在包装箱上后交给食品厂家。

此外，消费者购买产品时，需要手机扫码或编辑码号发短信，即可随时查询产品信息与质量认证等信息，并可及时举报虚假、错误信息。



Page 256

> 餐厅应用

手机电子菜单：

通过将餐饮店中的菜品介绍、餐饮文化等信息录入电子菜单中，用户通过扫码获得该手机网站的跳转链接获取商家相关信息。



Page 257

> 景点门票应用

景点门票、火车票等告别传统的文字纸张模式，采用二维码进行售票、检票等，可以大幅度提高通行效率，并且防止仿票。



Page 258

> 车辆管理应用

行驶证、驾驶证、车辆的年审文件等都印制有二维码，可以将有关车辆上的基本信息都保存在二维码中，信息的数字化便于管理部门的管理网络实施实时监控。



Page 259

> 课堂思政

上述举的这些例子，都是二维码真正应用于生活中的，并且随着技术的不断发展、移动设备的快速普及，二维码在生活中得到了越来越广泛的应用。你能想到的还有哪些非常普及的二维码应用方式呢？

古往今来，有许多的科研成果都投入到实际之中，并且获得广泛、实用的良好效果。当代大学生在平时也要坚持解决思想问题和解决实际问题相结合的思想，既要重视道理又要办实事，从而完成自身的全方位发展，并且能够为建设中国现代化社会主义强国作出贡献。

Page 260

二维码的制作

二维码是由很多0、1组成的数字矩阵。二维码是用某种特定的几何图形按一定规律在平面（二维方向上）分布的黑白相间的图形记录数据符号信息的；在代码编制上巧妙地利用构成计算机内部逻辑结构基础的“0”、“1”比特流的概念，通过图像输入设备或光电扫描设备自动识读以实现信息自动处理。同时它具有条码技术的一些共性。二维码生成器的制作需要一个二维码生成算法，或者一个二维码插件，然后用JAVA，C#，VB等编程语言编写一个调用软件就可以做成二维码生成器了。

二维码的代码实现-java

```
import java.awt.Color;
import java.awt.Graphics2D;
import java.awt.Image;
import java.awt.image.BufferedImage;
import java.io.File;
import javax.imageio.ImageIO;
import com.swetake.util.Qrcode;

public static void getQrcodeImage(String content, String imgPath)
{
    int width = 235;
    int height = 235;
    // 实例化一个对象
    Qrcode qrcode = new Qrcode();
    // 编码方式
    qrcode.setQrcodeMode('W');
```

二维码的代码实现-java

```
// 设置分辨率
qrcode.setQrcodeErrorCorrect('M');
// 二维码的版本
qrcode.setQrcodeVersion(15);
// 绘制二维码
// 画板
BufferedImage image = new BufferedImage(width, height,
    BufferedImage.TYPE_INT_RGB);
// 画板
Graphics2D gs = image.createGraphics();
// 设置背景颜色 白色
gs.setBackground(Color.white);
// 设置二维码的颜色
gs.setColor(Color.black);
```

二维码的代码实现-java

```
// 创建一个二维码的绘制区域
gs.clearRect(0, 0, width, height);
byte[] codeOut;
try {
    codeOut = content.getBytes("utf-8");
    boolean[] code =
    qrcode.callQrcode(codeOut);
    for (int i = 0; i < code.length; i++) {
        for (int j = 0; j < code.length; j++) {
            if (code[i][j]) {
                gs.fillRect(i * 3 + 2, j * 3 + 2, 3, 3);
            }
        }
    }
}
```

二维码的代码实现-java

```
// 加载logo D:/111.png 这个你需要改成你自己的
File file = new File("D:/111.png");
Image srcImage = ImageIO.read(file);
int imgWidth = srcImage.getWidth(null);
int imgHeigh = srcImage.getHeight(null);
gs.drawImage(srcImage, 83, 83, imgWidth, imgHeigh, null);
// 释放资源
gs.dispose();
image.flush();
```

二维码的代码实现-java

```
// 写入指定路径
ImageIO.write(image, "png", new File(imgPath));
System.out.println("二维码生成成功了！");
} catch (Exception e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}

public static void main(String[] args) {
    getQrcodeImage("hello!", "d/xxx.png");
}
```